NoSQL avec MongoDB

Cours 8 - Nettoyage des données

Steve Lévesque, Tous droits reservés © où applicables

Table des matières

1 Utilité du nettoyage des données

2 Définition et opérations populaires

3 Valeurs vides

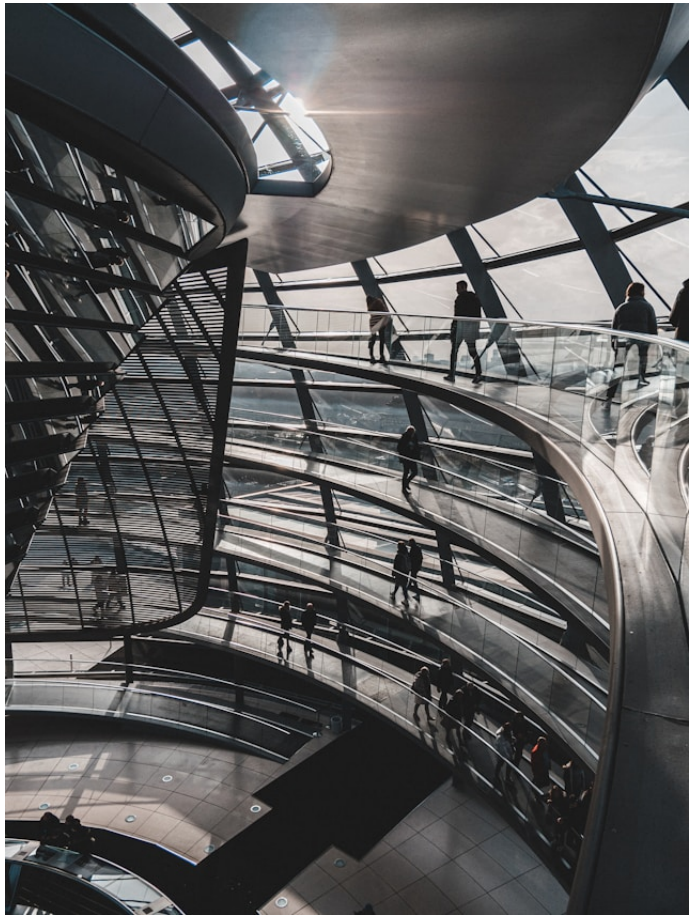
4 Transformation logarithmique

5 Normalisation

Utilité du nettoyage des données

Les données sont l'artéfact le **plus important** lorsqu'on entreprend un projet d'Intelligence Artificielle (AI).

Il arrive souvent que les meilleurs modèles de "Machine Learning" (ML) performent moins bien que des algorithmes de base d'Intelligence Aritificelle pour la simple raison que les données n'étaient pas de bonne qualité.



Définition et opérations populaires

Le nettoyage des données consiste à faire des opérations sur les données pour réduire ou régler les défauts existants. Les collectes de données ont **toujours un degré d'erreur associé**.

De plus, les divers modèles nécessitent des données pouvant demander des spécifications propres à respecter que des données brutes n'ont possiblement pas.

Définition et opérations populaires

Voici une liste simple* des opérations de nettoyage que nous allons voir :

- Remplacement des valeurs vides/"NaN"/"Null" par la moyenne ou des zéros.
- Transformation logarithmique (données en histogramme).
- Normalisation des données entre 0 et 1.

*il existe beaucoup plus d'opérations.

Valeurs vides

Ce nettoyage permet de remplacer les valeurs vides des attributs des documents respectifs par du contenu plus approprié lors de l'entraînement de modèles.

La valeur moyenne est généralement utilisée, puisqu'elle permet de garder l'uniformité des données d'une manière simple. Dépendamment du domaine d'affaires, une autre règle pourrait être choisie (i.e. zéros, maximum, minimum, etc.).

Valeurs vides

Listing: https://medium.com/@stevencibambo/loading-data-from-mongodb-with-pymongo-for-analysis-c0f61a8538a0

```
# Steve Levesque, All rights reserved where applicable
import pandas as pd
from pymongo import MongoClient

client = MongoClient()
db = client.PokemonJSONSchema
collection = db.PokemonsValidated
documents = collection.find()

df = pd.DataFrame(documents)

# Handle missing values, using the mean for numerical is recommended.
df["Total"] = df["Total"].fillna(int(df["Total"].mean()))
df = df.fillna(0)

print(df.iloc[0:2].to_string())
document_count = collection.count_documents({})
dataframe_count = len(df.index)
print(document_count, dataframe_count)
```

Transformation logarithmique

Ce nettoyage permet de rendre les données uniformes en termes de distribution des entrées, permettant généralement de rendre les modèles statistiques plus performants.

Transformation logarithmique

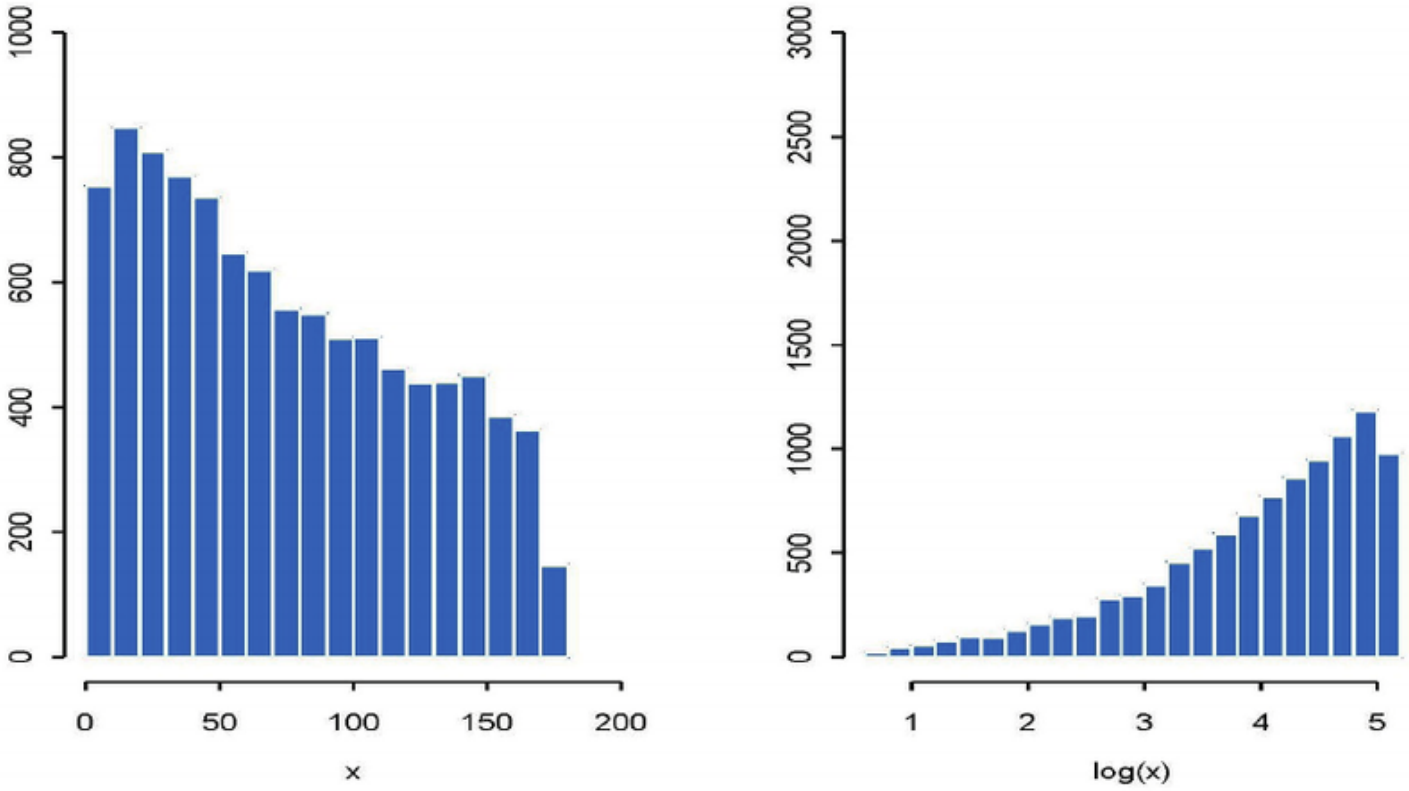


Figure: https://www.researchgate.net/figure/Histograms-of-original-data-left-plot-and-log-transformed-data-right-plot-from-a_fig1_264500976

Transformation logarithmique

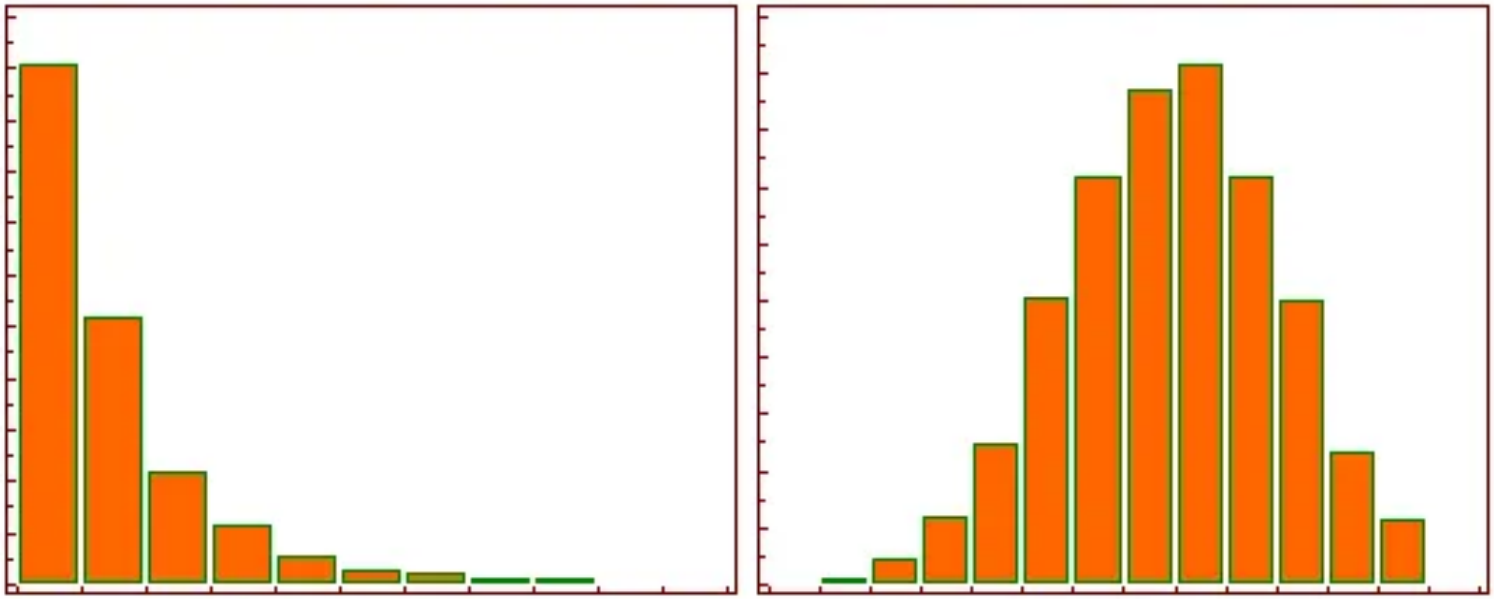


Figure:
https://medium.com/@kyawsawhtoon/log-transformation-purpose-and-interpretation-9444b4b049c9

Transformation logarithmique

Listing: https://medium.com/@stevencibambo/loading-data-from-mongodb-with-pymongo-for-analysis-c0f61a8538a0

```
# Steve Levesque, All rights reserved where applicable
import matplotlib.pyplot as plt, pandas as pd, numpy as np
from pymongo import MongoClient

column = 'Attack'
client = MongoClient()
db = client.admin
collection = db.Pokemons

df_ori = pd.DataFrame(collection.find())
data = np.array([document[column] for document in collection.find()])

# Apply logarithmic transformation
transformed_data = np.log(data)
df_new = pd.DataFrame(transformed_data)

df_ori.hist()
df_new.hist()
plt.show()
```

Normalisation

Ce nettoyage permet aux modèles de mieux performer puisque les données sont uniformément rapprochées dans une étendue entre 0 et 1.

Par exemple, la disparité (distance) entre l'âge et le salaire est grande au niveau numérique. Les modèles, surtout ceux qui sont sensibles à la notion de distance, sont fortement influencés par une telle distribution des données.

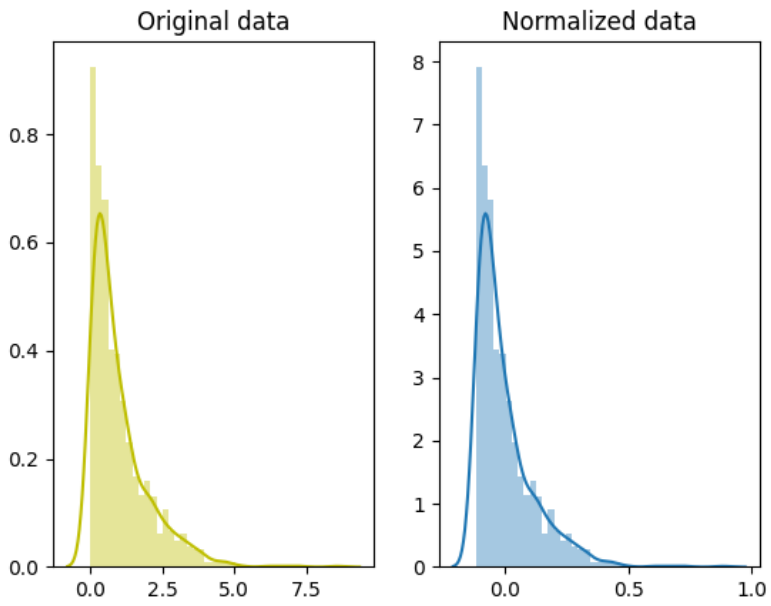


Figure: La topologie des données reste la même.

Normalisation

Listing: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html

```
# Steve Levesque, All rights reserved where applicable
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from pymongo import MongoClient

client = MongoClient()
db = client.admin
collection = db.Pokemons
documents = collection.find()

df = pd.DataFrame(documents)

# Normalize numeric fields
scaler = MinMaxScaler()
df[['Total', 'HP']] = scaler.fit_transform(df[['Total', 'HP']])

print(df[['Total', 'HP']])
print(df[['Total', 'HP']].min())
print(df[['Total', 'HP']].max())
```

Bibliographie

- https://medium.com/@stevencibambo/loading-data-from-mongodb-with-pymongo-for-analysis-c0
- https://medium.com/@kyawsawhtoon/log-transformation-purpose-and-interpretation-9444b4b0
- https://www.researchgate.net/figure/Histograms-of-original-data-left-plot-and-log-transfor fig1_264500976
- https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html